

第三单元

一、常用组件

1. 路由器 (Router)
2. <Routes>和 <Route>
3. <Link>
4. <NavLink>
5. <Outlet>
6. <Navigate>

二、常用 Hook

1. useRoutes
2. useNavigate
3. useLocation
4. useParams
5. useSearchParams

一、常用组件

1. 路由器 (Router)

路由器是路由系统的核心，包裹整个应用，提供路由上下文。

主要类型：

- `BrowserRouter` – 使用 HTML5 history API
- `HashRouter` – 使用 URL hash

2. <Routes>和 <Route>

```
1 import { Routes, Route } from 'react-router-dom';
2
3 function App() {
4   return (
5     <Routes>
6       {/* 基本路由 */}
7       <Route path="/" element={<Home />} />
8       <Route path="/about" element={<About />} />
9
10      {/* 嵌套路由 */}
11      <Route path="/dashboard" element={<DashboardLayout />>
12        <Route index element={<DashboardHome />} />
13        <Route path="profile" element={<Profile />} />
14        <Route path="settings" element={<Settings />} />
15      </Route>
16
17      {/* 动态路由 */}
18      <Route path="/users/:userId" element={<UserDetail />} />
19      <Route path="/products/:category/:id" element={<ProductDetail />} />
20
21      {/* 404 路由 */}
22      <Route path="*" element={<NotFound />} />
23    </Routes>
24  );
25 }
```

3. <Link>

作用：声明式导航链接

```
1 import { Link } from 'react-router-dom';
2
3 function Navigation() {
4   return (
5     <nav>
6       <Link to="/">首页</Link>
7       <Link to="/about">关于</Link>
8     </nav>
9   );
10 }
```

4. <NavLink>

作用：带激活状态的导航链接

```
1  import { NavLink } from 'react-router-dom';
2
3  function NavBar() {
4    return (
5      <nav>
6        {/* 基本用法 */}
7        <NavLink to="/">首页</NavLink>
8
9        {/* 自定义激活类名 */}
10       <NavLink
11         to="/about"
12         className={({ isActive }) =>
13           isActive ? 'nav-link active' : 'nav-link'
14         >
15         关于
16       </NavLink>
17
18       {/* 自定义样式 */}
19       <NavLink
20         to="/products"
21         style={({ isActive }) => ({
22           color: isActive ? 'red' : 'black',
23           fontWeight: isActive ? 'bold' : 'normal',
24           textDecoration: isActive ? 'underline' : 'none'
25         })}
26       >
27       产品
28     </NavLink>
29
30     {/* 精确匹配 */}
31     <NavLink
32       to="/dashboard"
33       end // 只有完全匹配 /dashboard 时才激活
34     >
35     仪表板
36   </NavLink>
37 </nav>
38 );
39 }
```

5. <Outlet>

作用：嵌套路由的渲染出口

```
1  import { Outlet, Link } from 'react-router-dom';
2
3  function MainLayout() {
4    return (
5      <div className="layout">
6        <header>
7          <h1>我的应用</h1>
8          <nav>
9            <Link to="/">首页</Link>
10           <Link to="/about">关于</Link>
11           <Link to="/dashboard">仪表板</Link>
12         </nav>
13       </header>
14
15       <main>
16         { /* 子路由将在这里渲染 */ }
17         <Outlet />
18       </main>
19
20       <footer>
21         <p>© 2024 版权所有</p>
22       </footer>
23     </div>
24   );
25 }
```

6. <Navigate>

作用：编程式重定向

```
1 import { Navigate } from 'react-router-dom';
2
3 // 1. 重定向到登录页
4 function PrivateRoute({ children, isAuthenticated }) {
5   if (!isAuthenticated) {
6     return <Navigate to="/login" replace />;
7   }
8   return children;
9 }
```

二、常用 Hook

1. useRoutes

作用：使用对象定义路由

```
1  import { useRoutes } from 'react-router-dom';
2
3  function App() {
4    const routes = useRoutes([
5      {
6        path: '/',
7        element: <Layout />,
8        children: [
9          { index: true, element: <Home /> },
10         { path: 'about', element: <About /> },
11         {
12           path: 'products',
13           element: <Products />,
14           children: [
15             { index: true, element: <ProductList /> },
16             { path: ':id', element: <ProductDetail /> }
17           ]
18         },
19         { path: '*', element: <NotFound /> }
20       ]
21     }
22   ]);
23
24   return routes;
25 }
```

2. useNavigate

作用：编程式导航

```
1 import { useNavigate } from 'react-router-dom';
2
3 function ProductCard({ product }) {
4   const navigate = useNavigate();
5
6   // 1. 基本跳转
7   const handleView = () => {
8     navigate(`/products/${product.id}`);
9   };
10
11   // 2. 跳转并传递状态
12   const handleEdit = () => {
13     navigate(`/products/${product.id}/edit`, {
14       state: {
15         product,
16         from: 'product-list'
17       }
18     });
19   };
20
21   // 3. 替换当前历史记录
22   const handleCheckout = () => {
23     navigate('/checkout', {
24       replace: true,
25       state: { product }
26     });
27   };
28
29   // 4. 返回上一页
30   const handleBack = () => {
31     navigate(-1);
32   };
33
34   // 5. 前进
35   const handleForward = () => {
36     navigate(1);
37   };
38
39   // 6. 相对路径跳转
40   const handleRelated = () => {
41     navigate(`../category/${product.category}`);
42   };
43
44   // 7. 获取当前位置
45   const getCurrentPath = () => {
```

```
46     console.log('当前在:', window.location.pathname);
47 };
48
49 return (
50     <div className="product-card">
51         <h3>{product.name}</h3>
52         <button onClick={handleView}>查看</button>
53         <button onClick={handleEdit}>编辑</button>
54         <button onClick={handleBack}>返回</button>
55     </div>
56 );
57 }
```

3. useLocation

作用：获取当前路由信息


```
1 import { useLocation } from 'react-router-dom';
2
3 function DebugComponent() {
4   const location = useLocation();
5
6   console.log('位置对象:', location);
7   /*
8     {
9       pathname: "/dashboard/profile", // 路径
10       search: "?tab=settings",       // 查询字符串
11       hash: "#section-2",           // hash
12       state: { from: "home" },      // 状态
13       key: "abc123"                 // 唯一标识
14     }
15   */
16
17   return (
18     <div>
19       <h2>当前位置信息:</h2>
20       <pre>{JSON.stringify(location, null, 2)}</pre>
21
22       <h3>解析信息:</h3>
23       <ul>
24         <li>路径: {location.pathname}</li>
25         <li>查询参数: {location.search}</li>
26         <li>Hash: {location.hash}</li>
27         <li>状态: {JSON.stringify(location.state || {})}</li>
28       </ul>
29     </div>
30   );
31 }
```

4. useParams

作用：获取动态路由参数

```
1 import { useParams } from 'react-router-dom';  
2  
3 // 1. 基本用法  
4 function UserDetails() {  
5   const { userId } = useParams();  
6   return <h1>用户 ID: {userId}</h1>;  
7 }
```

5. useParams

作用：读写 URL 查询参数

```
1 import { useSearchParams } from 'react-router-dom';
2
3 function ProductFilters() {
4   const [searchParams, setSearchParams] = useSearchParams();
5
6   // 获取参数
7   const category = searchParams.get('category') || '';
8   const page = searchParams.get('page') || '1';
9   const sort = searchParams.get('sort') || 'relevance';
10  const ratings = searchParams.getAll('rating') || [];
11
12  // 设置参数
13  const setCategory = (newCategory) => {
14    const params = Object.fromEntries(searchParams.entries());
15    setSearchParams({
16      ...params,
17      category: newCategory,
18      page: '1' // 重置页码
19    });
20  };
21
22  // 更新参数
23  const updateFilter = (key, value) => {
24    const params = Object.fromEntries(searchParams.entries());
25
26    if (value === '' || value === null) {
27      delete params[key];
28    } else {
29      params[key] = value;
30    }
31
32    setSearchParams(params);
33  };
34
35  // 删除参数
36  const removeFilter = (key) => {
37    const params = Object.fromEntries(searchParams.entries());
38    delete params[key];
39    setSearchParams(params);
40  };
41
42  // 清除所有筛选
43  const clearFilters = () => {
44    setSearchParams({});
45  };
```

```
46
47
48   return (
49     <div>
50       <select
51         value={category}
52         onChange={(e) => setCategory(e.target.value)}
53       >
54         <option value="">全部分类</option>
55         <option value="electronics">电子产品</option>
56         <option value="clothing">服装</option>
57         <option value="books">图书</option>
58       </select>
59
60       <button onClick={() => updateFilter('sort', 'price-asc')}>
61         按价格排序
62       </button>
63
64       <button onClick={clearFilters}>清除筛选</button>
65     </div>
66   );
}
```